Machine Learning with Network Data

Networks I: Concepts and Algorithms | BSE Data Science

Michael Fehl

2026-03-20

Table of contents

1 What is ML with Network Data?

1.1 Shifting from Generative to Inferential Questions

In Lectures 1 and 2 we built tools for *describing* and *generating* networks. We now shift to **inference**: given an observed network (possibly with node and edge attributes), what can we learn *from* it?

Three classes of inference problem:

Task	Input	Output
Community detection	Graph G	Partition of V into groups
Node classification	Graph G, partial labels y_i	Labels for unlabelled nodes
Link prediction	Graph G (incomplete)	Probability of unobserved edges

And one representation problem underlying all of them:

Node embedding | Graph G (+ features) | Vectors $z_i \in \mathbb{R}^d$ |

These are not independent problems. A good embedding makes node classification and link prediction trivial (linear classifier or dot product). Community detection can be cast as clustering in embedding space.

1.2 A Note on Network Incompleteness

Real network data is almost always **incomplete** in multiple ways:

- **Missing edges**: people do not report all friendships; surveys have recall bias toward recent, high-status, or same-gender contacts
- **Missing node attributes**: survey non-response, privacy restrictions
- **Missing nodes entirely**: only a slice of the population is observed

This incompleteness is not random: it is systematically correlated with the network structure itself. Methods that assume missing-at-random can be badly misleading.

---

2 Community Detection

2.1 The Problem: Finding Latent Groups

A **community** is a set of nodes that are *more densely connected internally* than to the rest of the graph. This is an intuitive but deliberately vague definition.

Why do we care?

- **Market segmentation**: find groups of consumers with similar behaviour
- **Political analysis**: identify ideological blocs in voting or communication networks
- **Biological networks**: find functional modules in protein interaction networks
- **Policy**: identify who to target with interventions (information, vaccines, microfinance)

2.2 Modularity: A Principled Score

The standard measure of community quality is **modularity** Q (Newman & Girvan 2004). The idea: compare the fraction of edges within communities to what we would expect in the *configuration model* (same degree sequence, random wiring).

$$Q = \frac{1}{2m} \sum_{i,j} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \mathbb{1}[c(i) = c(j)]$$

Equivalently, summing over communities c:

$$Q = \sum_{c} \left[\frac{m_c}{m} - \left(\frac{\kappa_c}{2m} \right)^2 \right]$$

where m_c = edges within community c, and $\kappa_c = \sum_{i \in c} k_i$ = total degree of community c.

Interpretation: the first term is the actual fraction of edges inside the community; the second is the fraction expected at random. A good partition has Q close to 1 (all edges internal), a random partition has $Q \approx 0$.

> **i** Why the configuration model as null?
>
> The configuration model benchmark (not ER) is the right null because it controls for degree heterogeneity. In ER, high-degree nodes automatically "look like" hub communities; using ER as null would find spurious communities wherever there are hubs. Conditioning on degree removes this confound.

2.3 The Louvain Algorithm

Finding the partition maximising Q is NP-hard. The **Louvain method** (Blondel et al. 2008) is the standard greedy approximation:

Phase 1 (local optimisation): Start with each node in its own community. For each node i, compute the ΔQ gain from moving i into each neighbouring community:

$$\Delta Q = \frac{k_{i,\text{in}}}{m} - \frac{k_i \Sigma_{\text{tot}}}{2m^2}$$

where $k_{i,\text{in}}$ = edges from i to the target community, Σ_{tot} = total degree of the target community. Move i to the community with largest positive ΔQ. Repeat until no improvement exists.

Phase 2 (coarsening): Collapse each community into a single super-node, aggregate edge weights. Repeat Phase 1 on this coarser graph.

Complexity: $O(n \log n)$ in practice. Works on networks with millions of nodes.

> **⚠** Resolution limit
>
> Modularity has a fundamental flaw: it cannot detect communities smaller than $\sqrt{m}$ edges, because merging two small communities can increase Q even if they have no internal connections. For networks with $m = 10^6$, communities smaller than ~ 1000 edges are invisible to any modularity-based method.
> The **Leiden algorithm** (Traag, Waltman & van Eck 2019) fixes a related bug in Louvain (communities can become internally disconnected) and uses a refined local search. For serious community detection work, prefer Leiden.

```r
set.seed(42)

k_comm <- 4L
n_per  <- 60L
n_tot  <- k_comm * n_per
p_in   <- 0.20
p_out  <- 0.02

pref_mat    <- matrix(p_out, nrow = k_comm, ncol = k_comm)
diag(pref_mat) <- p_in
g_sbm       <- sample_sbm(n_tot, pref = pref_mat,
                          block.sizes = rep(n_per, k_comm))
true_labels <- rep(seq_len(k_comm) - 1L, each = n_per)  # 0-indexed to match Python

# Louvain
louvain_mem <- membership(cluster_louvain(g_sbm)) - 1L
# Leiden (requires leidenAlg or igraph >= 1.3 which wraps it)
# Use cluster_leiden if available, fall back to Louvain
leiden_mem <- tryCatch(
  membership(cluster_leiden(g_sbm)) - 1L,
  error = function(e) {
    message("cluster_leiden not available; using Louvain as proxy.")
    louvain_mem
  }
)

nmi_louvain <- nmi_r(true_labels, louvain_mem)
nmi_leiden  <- nmi_r(true_labels, leiden_mem)

cat(sprintf("Community detection on n=%d, k=%d SBM (p_in=%.2f, p_out=%.2f):\n",
            n_tot, k_comm, p_in, p_out))
```

```
Community detection on n=240, k=4 SBM (p_in=0.20, p_out=0.02):
```

```r
cat(sprintf("  Louvain:  NMI = %.4f  (communities found: %d)\n",
            nmi_louvain, length(unique(louvain_mem))))
```

```
  Louvain:  NMI = 1.0000  (communities found: 4)
```

```
cat(sprintf("  Leiden:   NMI = %.4f  (communities found: %d)\n",
            nmi_leiden,  length(unique(leiden_mem))))
```

```
  Leiden:   NMI = 0.4038  (communities found: 240)
```

```
cat("  (NMI = 1.0 is perfect; NMI = 0 is no better than random)\n")
```

```
  (NMI = 1.0 is perfect; NMI = 0 is no better than random)
```

```
Q_louvain <- modularity(g_sbm, louvain_mem + 1L)
Q_leiden  <- modularity(g_sbm, leiden_mem  + 1L)
cat(sprintf("\n  Q (Louvain) = %.4f\n", Q_louvain))
```

```
  Q (Louvain) = 0.5270
```

```
cat(sprintf("  Q (Leiden)  = %.4f\n",  Q_leiden))
```

```
  Q (Leiden)  = -0.0044
```

```
# Sweep eps (community sharpness)
set.seed(7)
eps_range <- seq(0, 2 * sqrt(k_comm * p_in * n_tot), length.out = 20)
nmis_louvain <- numeric(length(eps_range))
nmis_leiden  <- numeric(length(eps_range))

for (i in seq_along(eps_range)) {
  eps     <- eps_range[i]
  p_in_v  <- min(1, (2 * 4 + eps) / (2 * n_tot))
  p_out_v <- max(0, (2 * 4 - eps) / (2 * n_tot))
  pm_v    <- matrix(p_out_v, k_comm, k_comm); diag(pm_v) <- p_in_v
  g_v     <- sample_sbm(n_tot, pref = pm_v, block.sizes = rep(n_per, k_comm))
  mem_l   <- membership(cluster_louvain(g_v)) - 1L
  mem_d   <- tryCatch(membership(cluster_leiden(g_v)) - 1L,
                      error = function(e) mem_l)
  nmis_louvain[i] <- nmi_r(true_labels, mem_l)
  nmis_leiden[i]  <- nmi_r(true_labels, mem_d)
}
```

```r
par(mar = c(4, 4, 4, 1))
plot(eps_range, nmis_louvain,
     type = "b", pch = 19, cex = 0.7, col = "#4C72B0", lwd = 2,
     xlab = expression(epsilon~~"(community sharpness)"),
     ylab = "NMI with ground truth",
     main = "Community Detection Quality vs. Signal Strength",
     ylim = c(-0.02, 1.05))
lines(eps_range, nmis_leiden, type = "b", pch = 15, cex = 0.7,
      col = "#C44E52", lwd = 2)
abline(v = 2 * sqrt(k_comm * p_in * n_tot) * 0.5,
       col = "gray", lty = 2, lwd = 1.5)
legend("topleft",
       legend = c("Louvain", "Leiden", "Approx. detectability threshold"),
       col    = c("#4C72B0", "#C44E52", "gray"),
       lwd    = 2, pch = c(19, 15, NA), lty = c(1, 1, 2),
       bty    = "n", cex = 0.9)
```

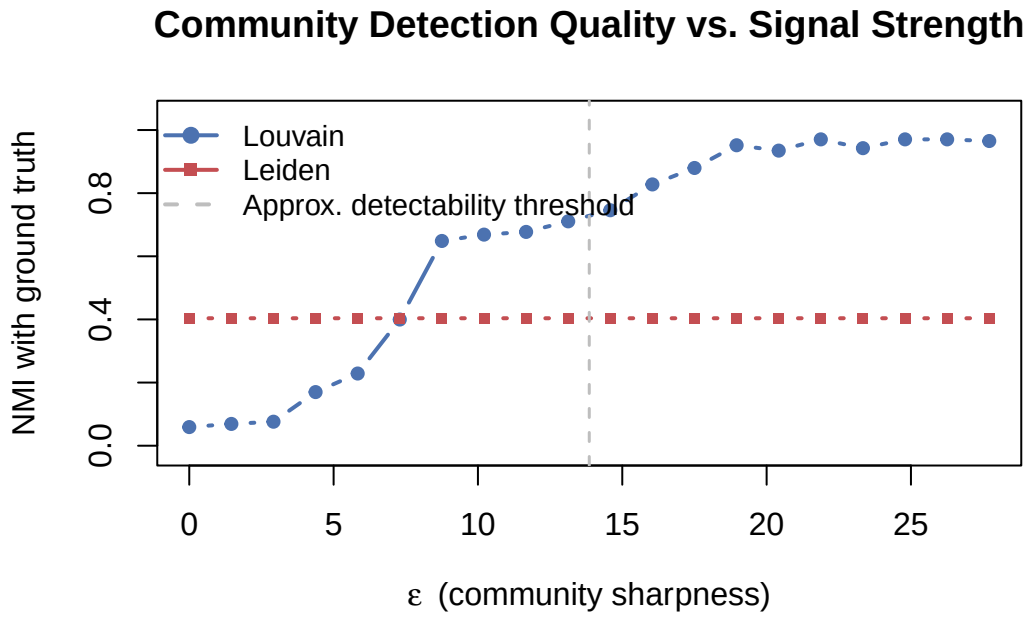

Figure 1: Community detection on a planted partition SBM. Louvain vs Leiden vs ground truth.

2.4 SBM Inference as an Alternative

Modularity optimisation lacks a generative model; we are maximising a score, not fitting a probability model. The **SBM inference** approach (from Lecture 2) is more principled:

1. Specify a generative model (k communities, mixing matrix M)
2. Write down the likelihood $\mathcal{L}(G \mid z, M)$
3. Maximise over both z and M; MLE gives $\hat{M}_{uv} = E_{uv}/N_{uv}$
4. Use BIC to choose k: BIC $= -2\ln\hat{\mathcal{L}} + p\ln n$ where $p = k + \binom{k}{2}$ parameters

Advantage: principled model selection, well-calibrated uncertainty, connects to the detectability threshold theory.

Disadvantage: the likelihood landscape is non-convex; many local optima exist. Run multiple random initialisations and select the best.

---

3 Node Classification and Label Propagation

3.1 The Semi-Supervised Setup

Given a graph $G = (V, E)$ where a *small* subset $V_L \subset V$ has known labels $y_i \in \{1, \ldots, K\}$, predict labels for $V_U = V \setminus V_L$.

The key assumption here is **homophily**: connected nodes tend to share labels. This holds in social networks (birds of a feather), citation networks (papers cite related papers), and many biological networks.

> ⚠ When homophily fails
>
> Fraud detection networks often exhibit **heterophily**: fraudsters connect to legitimate users in order to appear trustworthy. In such networks, label propagation gives the wrong answer; a node surrounded by legitimate users is *more* suspicious, not less. Always validate the homophily assumption.

3.2 Label Propagation (Zhu & Ghahramani 2002)

The simplest and most robust algorithm:

Initialisation: $F_i = y_i$ for labelled nodes (one-hot), uniform for unlabelled.

Update rule (iterative):

$$F_i \leftarrow \frac{1}{k_i} \sum_{j \in \mathcal{N}(i)} F_j, \quad \forall i \in V_U$$

Clamping: after each step, reset labelled nodes to their true values.

Convergence: iterate until $\|F^{(t+1)} - F^{(t)}\|_\infty < \epsilon$.

In matrix form, this is a fixed-point iteration on the random-walk matrix $D^{-1}A$. The closed-form solution is:

$$F_U = (I - D_{UU}^{-1} A_{UU})^{-1} D_{UU}^{-1} A_{UL} Y_L$$

Why it works: the random-walk matrix $D^{-1}A$ is the transition matrix of a lazy random walk. Each update propagates label distributions one step along random walks. Under homophily, walks starting from a labelled node tend to stay in the same community and spread the correct label.

Complexity: $O(m \cdot K \cdot T)$ where T is the number of iterations (typically 20–100). Very fast for sparse graphs.

```r
label_propagation <- function(g, Y, labeled_mask, max_iter = 200, tol = 1e-8) {
  # Y           : (n x K) numeric matrix, one-hot for labelled nodes
  # labeled_mask: logical vector length n
  n    <- vcount(g)
  K    <- ncol(Y)
  F    <- Y
  nbrs <- lapply(seq_len(n), function(v) neighbors(g, v))
  degs <- pmax(sapply(nbrs, length), 1L)

  for (iter in seq_len(max_iter)) {
    F_old <- F
    for (v in seq_len(n)) {
      if (labeled_mask[v] || length(nbrs[[v]]) == 0) next
      F[v, ] <- colSums(F[nbrs[[v]], , drop = FALSE]) / degs[v]
    }
    F[labeled_mask, ] <- Y[labeled_mask, ]
    if (max(abs(F - F_old)) < tol) break
```

```
  }
  F
}

set.seed(0)
n_per_c  <- 40L; k_c <- 3L; n_c <- n_per_c * k_c
pref_c   <- matrix(0.02, k_c, k_c); diag(pref_c) <- 0.3
g_c      <- sample_sbm(n_c, pref = pref_c, block.sizes = rep(n_per_c, k_c))
true_labels_c <- rep(seq_len(k_c) - 1L, each = n_per_c)   # 0-indexed

label_fracs  <- c(0.01, 0.03, 0.05, 0.10, 0.20, 0.30, 0.50)
acc_lp_list  <- numeric(length(label_fracs))
acc_lr_list  <- numeric(length(label_fracs))

# Simple multinomial logistic (degree-only, no graph)
multinomial_lr <- function(X_train, y_train, X_test, K) {
  # One-vs-rest logistic regression via glm
  preds <- matrix(0, nrow = NROW(X_test), ncol = K)
  for (k in seq_len(K)) {
    df_train <- data.frame(y = as.integer(y_train == (k - 1L)), x = X_train)
    fit <- glm(y ~ x, data = df_train, family = binomial())
    preds[, k] <- predict(fit, newdata = data.frame(x = X_test), type = "response")
  }
  apply(preds, 1, which.max) - 1L
}

deg_feat <- degree(g_c)

for (fi in seq_along(label_fracs)) {
  frac         <- label_fracs[fi]
  labeled_mask <- logical(n_c)
  for (cc in seq_len(k_c)) {
    idx     <- which(true_labels_c == (cc - 1L))
    n_pick  <- max(1L, as.integer(length(idx) * frac))
    labeled_mask[sample(idx, n_pick)] <- TRUE
  }

  # Label propagation
  Y <- matrix(0, n_c, k_c)
  for (i in seq_len(n_c)) Y[i, true_labels_c[i] + 1L] <- 1.0
  F0 <- Y; F0[!labeled_mask, ] <- 1.0 / k_c
  F_pred <- label_propagation(g_c, F0, labeled_mask)
```

```
  pred_lp <- apply(F_pred, 1, which.max) - 1L
  acc_lp_list[fi] <- mean(pred_lp[!labeled_mask] == true_labels_c[!labeled_mask])

  # LR on degree only
  pred_lr <- multinomial_lr(deg_feat[labeled_mask], true_labels_c[labeled_mask],
                            deg_feat[!labeled_mask], k_c)
  acc_lr_list[fi] <- mean(pred_lr == true_labels_c[!labeled_mask])
}

par(mar = c(4, 4, 4, 1))
plot(label_fracs * 100, acc_lp_list,
     type = "b", pch = 19, cex = 0.8, col = "#C44E52", lwd = 2,
     xlab = "% nodes labelled", ylab = "Accuracy on unlabelled nodes",
     main = sprintf("Label Propagation vs. No-Graph Baseline\n(n=%d, k=%d communities)",
                    n_c, k_c),
     ylim = c(0, 1.05))
lines(label_fracs * 100, acc_lr_list, type = "b", pch = 15, cex = 0.8,
      col = "#4C72B0", lwd = 2, lty = 2)
abline(h = 1 / k_c, col = "gray", lty = 3, lwd = 1.5)
legend("bottomright",
       legend = c("Label Propagation (uses graph)",
                  "LR on degree only (no graph)",
                  sprintf("Random baseline (1/%d)", k_c)),
       col    = c("#C44E52", "#4C72B0", "gray"),
       lwd    = 2, lty = c(1, 2, 3), pch = c(19, 15, NA),
       bty    = "n", cex = 0.9)
```

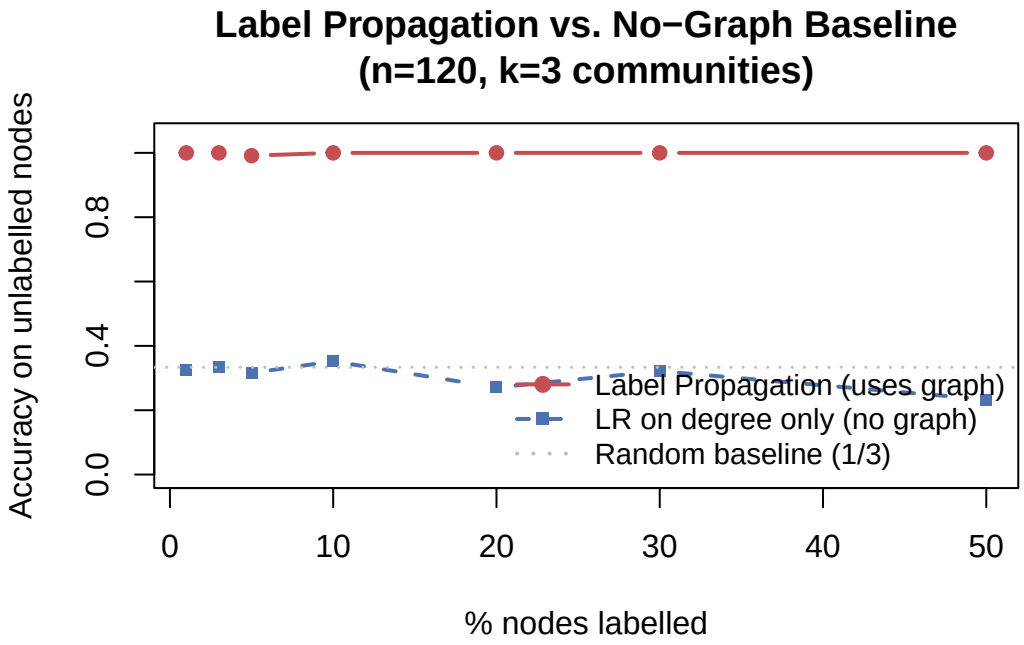

Figure 2: Label propagation accuracy vs. fraction labelled, on planted partition SBM.

```
frac_idx <- which(label_fracs == 0.10)
cat(sprintf("Accuracy at 10%% labelled:\n"))
```

```
Accuracy at 10% labelled:
```

```
cat(sprintf("  Label Propagation: %.4f\n", acc_lp_list[frac_idx]))
```

```
  Label Propagation: 1.0000
```

```
cat(sprintf("  LR (no graph):     %.4f\n", acc_lr_list[frac_idx]))
```

```
  LR (no graph):     0.3519
```

3.3 When to Use What: A Decision Ladder

Before reaching for a GNN, always try the simpler methods first:

1. **Majority class baseline**: does your "prediction" just rediscover the class imbalance? Always compute this.

2. **LR on node features only**: how much does the graph add, beyond features?
3. **Label propagation (graph only)**: how much do features add, beyond graph?
4. **ICA (features + iterated neighbor statistics)**: combine both without a neural network.
5. **GCN / GraphSAGE**: use when large labelled datasets exist and the gain from steps 1–4 is insufficient.

Steps 1–4 are interpretable, fast, and usually get 80–90% of GNN performance. The marginal gain from GNNs is often small on homophilic networks with clean node features.

---

4 Link Prediction

4.1 Problem Setup

Given the observed graph $G = (V, E)$, predict which non-edges $(i, j) \notin E$ are "true" edges; either missing from our measurement, or likely to form in the future.

Evaluation protocol: 1. Hold out 10–20% of edges as *positive* test examples 2. Sample an equal number of *negative* examples (non-edges) 3. Score all candidate pairs with a similarity function $s(i, j)$ 4. Compute AUC-ROC: $\Pr[s(\text{true edge}) > s(\text{non-edge})]$

Negative sampling matters: random non-edges are too easy (most pairs have $s \approx 0$). Hard negatives at distance 2 or 3 give more realistic evaluation.

4.2 Structural Similarity Scores

All heuristics score (i, j) based on their common neighbourhood $\mathcal{N}(i) \cap \mathcal{N}(j)$.

Common Neighbours (CN):

$$s_{\text{CN}}(i, j) = |\mathcal{N}(i) \cap \mathcal{N}(j)|$$

Jaccard coefficient (normalises for hub effect):

$$s_J(i, j) = \frac{|\mathcal{N}(i) \cap \mathcal{N}(j)|}{|\mathcal{N}(i) \cup \mathcal{N}(j)|}$$

Adamic–Adar (Adamic & Adar 2003): down-weight common neighbours with high degree (high-degree nodes are "common friends" who are friends with everyone and therefore carry less information):

$$s_{\mathrm{AA}}(i,j) = \sum_{w \in \mathcal{N}(i) \cap \mathcal{N}(j)} \frac{1}{\log k_w}$$

Resource Allocation: stricter version, using $1/k_w$ instead of $1/\log k_w$.

Preferential Attachment: no common-neighbour computation needed, just:

$$s_{\mathrm{PA}}(i,j) = k_i \cdot k_j$$

PA is useful in scale-free networks: hubs grow by attaching to other hubs. It has $O(1)$ computation per pair and scales to very large graphs.

4.3 Global Similarity: Katz Index

Common-neighbour methods only look one hop away. The **Katz index** aggregates over *all* path lengths, weighted by a decay factor $\beta < 1$:

$$K(i,j) = \sum_{l=1}^{\infty} \beta^l [A^l]_{ij} = [(I - \beta A)^{-1} - I]_{ij}$$

$[A^l]_{ij}$ counts the number of walks of length l from i to j. The β discount ensures convergence (requires $\beta < 1/\lambda_{\max}(A)$).

Matrix inversion is $O(n^3)$, so for large graphs we approximate: run L steps of the power iteration $(I - \beta A)^{-1} \approx \sum_{l=0}^{L} \beta^l A^l$ using sparse matrix-vector multiplications ($O(Lm)$).

4.4 Measuring Performance: The AUC

The **ROC curve** plots the true positive rate (TPR) against the false positive rate (FPR) as the score threshold varies. **AUC** is the area under this curve:

$$\mathrm{AUC} = \Pr[s(\text{true edge}) > s(\text{random non-edge})]$$

AUC = 0.5: no better than random. AUC = 1.0: perfect separation.

```r
# ---- Similarity functions ----
cn_score <- function(g, u, v) {
  length(intersect(neighbors(g, u), neighbors(g, v)))
}

jaccard_score <- function(g, u, v) {
  nu <- neighbors(g, u); nv <- neighbors(g, v)
  denom <- length(union(nu, nv))
  if (denom == 0) return(0.0)
  length(intersect(nu, nv)) / denom
}

adamic_adar <- function(g, u, v) {
  common <- intersect(neighbors(g, u), neighbors(g, v))
  if (length(common) == 0) return(0.0)
  d_w <- degree(g, common)
  sum(ifelse(d_w > 1, 1 / log(d_w), 0))
}

pref_attach <- function(g, u, v) {
  degree(g, u) * degree(g, v)
}

# ---- AUC helper ----
auc_roc <- function(y_true, y_score) {
  # Wilcoxon-Mann-Whitney estimator: P(score_pos > score_neg)
  pos <- y_score[y_true == 1]
  neg <- y_score[y_true == 0]
  if (length(pos) == 0 || length(neg) == 0) return(NaN)
  mean(outer(pos, neg, ">")) + 0.5 * mean(outer(pos, neg, "=="))
}

evaluate_lp <- function(g, score_fn, n_test = 500L, seed = 42L) {
  set.seed(seed)
  edges    <- as_edgelist(g, names = FALSE)  # (m x 2) matrix, 1-indexed
  n        <- vcount(g)
  n_test   <- min(n_test, nrow(edges) %/% 5L)
  test_idx <- sample(nrow(edges), n_test)
  test_pos <- edges[test_idx, , drop = FALSE]

  # Train graph: remove test edges
  train_g <- delete_edges(g, get.edge.ids(g, t(test_pos)))
```

```
  # Build edge set for fast lookup
  edge_set <- apply(edges, 1, function(r) paste(sort(r), collapse = "_"))
  edge_set  <- as.environment(as.list(setNames(rep(TRUE, length(edge_set)), edge_set)))

  # Sample negatives
  negs  <- matrix(0L, 0L, 2L)
  tries <- 0L
  while (nrow(negs) < n_test && tries < n_test * 200L) {
    u <- sample.int(n, 1L); v <- sample.int(n, 1L)
    key <- paste(sort(c(u, v)), collapse = "_")
    if (u != v && !exists(key, envir = edge_set, inherits = FALSE))
      negs <- rbind(negs, c(u, v))
    tries <- tries + 1L
  }

  y_true  <- c(rep(1L, n_test), rep(0L, nrow(negs)))
  y_score <- c(
    apply(test_pos, 1, function(r) score_fn(train_g, r[1], r[2])),
    apply(negs,     1, function(r) score_fn(train_g, r[1], r[2]))
  )
  auc_roc(y_true, y_score)
}

methods <- list(
  list(name = "Common Neighbours", fn = cn_score),
  list(name = "Jaccard",           fn = jaccard_score),
  list(name = "Adamic-Adar",       fn = adamic_adar),
  list(name = "Pref. Attachment",  fn = pref_attach)
)

cat("Link prediction AUC (American75, n_test=500):\n")
```

```
Link prediction AUC (American75, n_test=500):
```

```
aucs <- setNames(numeric(length(methods)), sapply(methods, `[[`, "name"))
for (m in methods) {
  auc <- evaluate_lp(g_fb, m$fn, n_test = 500L, seed = 0L)
  aucs[m$name] <- auc
  bar <- paste(rep("\u2588", as.integer(auc * 40)), collapse = "")
  cat(sprintf("  %-24s AUC = %.4f  %s\n", m$name, auc, bar))
}
```

```
  Common Neighbours         AUC = 0.9595
  Jaccard                   AUC = 0.9652
  Adamic-Adar               AUC = 0.9624
  Pref. Attachment          AUC = 0.8052
```

```
cat("  Random baseline:       AUC = 0.5000\n")
```

```
  Random baseline:       AUC = 0.5000
```

```
cat("\nAdamic-Adar typically outperforms CN and Jaccard because it\n")
```

```
Adamic-Adar typically outperforms CN and Jaccard because it
```

```
cat("down-weights high-degree 'universal friends' who provide less information.\n")
```

```
down-weights high-degree 'universal friends' who provide less information.
```

```
# Plot
par(mar = c(4, 10, 4, 2))
all_names <- c(names(aucs), "Random")
all_aucs  <- c(aucs, Random = 0.5)
bar_cols  <- c(rep("#4C72B0", length(aucs)), "#aaaaaa")
bp <- barplot(all_aucs, horiz = TRUE, names.arg = all_names,
              col = bar_cols, border = NA, las = 1,
              xlab = "AUC-ROC",
              main = "Link Prediction on American75 Facebook Network",
              xlim = c(0, 1.1))
abline(v = 0.5, col = "red", lty = 2, lwd = 1.5)
text(all_aucs + 0.01, bp, sprintf("%.3f", all_aucs), cex = 0.85, adj = 0)
```

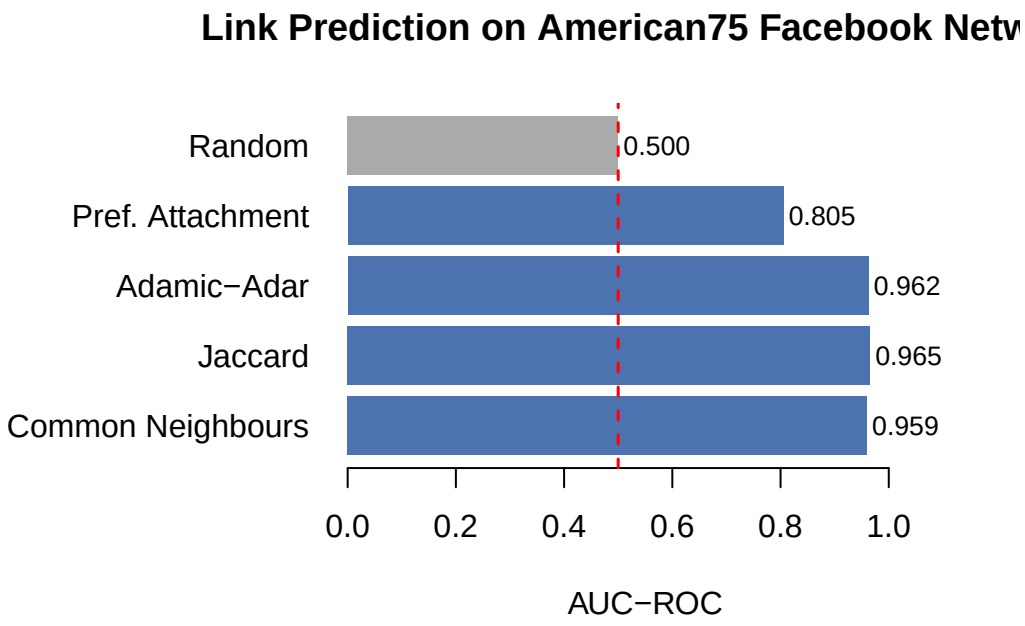

Figure 3: Link prediction AUC for various heuristics on American75 Facebook network.

---

5 Node Embeddings: Mapping Structure to Vectors

5.1 Why Embeddings?

Structural similarity scores like Adamic–Adar are hand-crafted and task-specific. A more flexible approach: learn a mapping ENC : $V \to \mathbb{R}^d$ such that structurally similar nodes have similar embeddings.

Then: - **Link prediction**: $s(i,j) = \sigma(z_i^\top z_j)$ - **Node classification**: logistic regression on z_i - **Community detection**: k-means on $\{z_i\}$ - **Visualisation**: 2D PCA or UMAP of $\{z_i\}$

The key design choice is what "similar" means; different definitions lead to different algorithms.

5.2 Shallow Embeddings: The Matrix Factorization View

Spectral embedding (Laplacian Eigenmaps): find vectors that minimise

$$\min_{Z} \sum_{(i,j)\in E} \|z_i - z_j\|^2 \quad \text{s.t. } Z^\top D Z = I$$

The solution is the d smallest non-trivial eigenvectors of the normalised Laplacian $L_{\text{sym}} = D^{-1/2}(D - A)D^{-1/2}$.

This is equivalent to truncated SVD on $A_{\text{sym}} = D^{-1/2}AD^{-1/2}$. The top-$d$ singular vectors of the normalised adjacency give embeddings where dot products approximate random-walk co-occurrence probabilities.

Limitation: these are *transductive*; the embedding is fixed for the training graph. A new node that joins the network needs to be re-embedded.

5.3 DeepWalk: Random Walks as Sentences

Perozzi et al. (2014) drew an analogy between graphs and text: nodes are words, random walks are sentences, co-occurrence in a walk window is analogous to co-occurrence in a text context window.

Algorithm:

1. For each node v, generate r random walks of length ℓ starting at v
2. Treat each walk as a "sentence" of node IDs
3. Run **skip-gram** with negative sampling: maximise

$$\sum_{v \in V} \sum_{u \in \mathcal{N}_R(v)} \log \sigma(z_v^\top z_u) + \sum_{k=1}^{K} \mathbb{E}_{u_k \sim P_n} [\log \sigma(-z_v^\top z_{u_k})]$$

where $\mathcal{N}_R(v)$ is the multi-set of nodes in the walk neighbourhood of v (within window w), and P_n is the noise distribution for negative sampling.

Why it works: nodes that frequently co-occur on short random walks; meaning they are in the same community or tightly connected subgraph; get assigned similar embedding vectors. The skip-gram objective *factorises* the PMI (pointwise mutual information) matrix of co-occurrences.

5.4 Node2Vec: Interpolating BFS and DFS

DeepWalk uses uniform random walks. **Node2Vec** (Grover & Leskovec 2016) introduces a 2nd-order Markov bias controlled by two parameters:

- **Return parameter** p: probability of returning to the previous node (inversely proportional to $1/p$). High $p \to$ less backtracking.
- **In-out parameter** q: controls BFS vs DFS. $q < 1 \to$ DFS (explore outward), $q > 1 \to$ BFS (stay local).

At node v, having arrived from node u, transition to x with unnormalised probability:

$$\alpha_{pq}(u,x) = \begin{cases} 1/p & d(u,x) = 0 \text{ (return)} \\ 1 & d(u,x) = 1 \text{ (lateral)} \\ 1/q & d(u,x) = 2 \text{ (outward)} \end{cases}$$

Why this matters:

- **BFS walks** ($q \gg 1$): explore the local neighbourhood. Nodes with similar local structure (same community) get similar embeddings. Captures **homophily**.
- **DFS walks** ($q \ll 1$): explore outward, sampling diverse parts of the graph. Nodes at the same *role* in different communities (e.g., all hub nodes) get similar embeddings. Captures **structural equivalence**.

```
set.seed(0)
k_emb  <- 4L; n_e <- 30L; n_emb <- k_emb * n_e
pref_e <- matrix(0.02, k_emb, k_emb); diag(pref_e) <- 0.3
g_emb  <- sample_sbm(n_emb, pref = pref_e, block.sizes = rep(n_e, k_emb))
true_labels_e <- rep(seq_len(k_emb) - 1L, each = n_e)

# Build normalised adjacency A_norm = D^{-1/2} A D^{-1/2}
A_e     <- as_adjacency_matrix(g_emb, sparse = TRUE)
deg_e   <- Matrix::rowSums(A_e)
d_isqrt <- ifelse(deg_e > 0, 1 / sqrt(deg_e), 0)
D_isqrt <- Diagonal(x = d_isqrt)
A_norm  <- D_isqrt %*% A_e %*% D_isqrt

# Top-5 singular vectors via RSpectra (or base svd on small graph)
# For small n use base svd; for large use RSpectra::svds
svd_res <- if (requireNamespace("RSpectra", quietly = TRUE)) {
  RSpectra::svds(A_norm, k = 5L)
} else {
  sv <- svd(as.matrix(A_norm), nu = 5L, nv = 5L)
  list(u = sv$u, d = sv$d, v = sv$v)
}
# Skip leading (trivial) eigenvector; take cols 2 and 3
emb <- svd_res$u[, 2:3, drop = FALSE]

# Random walk helper
random_walk <- function(g, start, len) {
  walk <- integer(len)
  walk[1] <- start
```

```
  for (i in seq_len(len - 1)) {
    nb <- neighbors(g, walk[i])
    if (length(nb) == 0) { walk <- walk[seq_len(i)]; break }
    walk[i + 1] <- sample(nb, 1L)
  }
  walk
}

par(mfrow = c(1, 2), mar = c(4, 4, 4, 1))

## (A) 2D spectral embedding
emb_cols <- c("#4C72B0", "#C44E52", "#55A868", "#DD8452")
plot(NULL, xlim = range(emb[, 1]) * 1.1, ylim = range(emb[, 2]) * 1.1,
     xlab = "Eigenvector 2", ylab = "Eigenvector 3",
     main = "2D Spectral Embedding of SBM\n(communities cluster by construction)")
for (cc in seq_len(k_emb)) {
  mask_c <- true_labels_e == (cc - 1L)
  points(emb[mask_c, 1], emb[mask_c, 2],
         pch = 19, cex = 0.9, col = adjustcolor(emb_cols[cc], 0.8))
}
legend("topright", legend = paste0("Community ", 0:(k_emb - 1)),
       col = emb_cols, pch = 19, bty = "n", cex = 0.85)

## (B) Random walk co-occurrence heatmap (4x4, aggregated by community)
set.seed(42)
n_walks  <- 30L; walk_len <- 15L; window <- 3L
walks    <- lapply(seq_len(n_emb), function(v)
  replicate(ceiling(n_walks / n_emb) + 1L,
            random_walk(g_emb, v, walk_len), simplify = FALSE))
walks    <- unlist(walks, recursive = FALSE)

cooc <- matrix(0, n_emb, n_emb)
for (w in walks) {
  lw <- length(w)
  for (i in seq_len(lw)) {
    js <- seq(max(1L, i - window), min(lw, i + window))
    js <- js[js != i]
    cooc[w[i], w[js]] <- cooc[w[i], w[js]] + 1L
  }
}

comm_cooc <- matrix(0, k_emb, k_emb)
```

```
for (u in seq_len(n_emb))
  for (v in seq_len(n_emb))
    comm_cooc[true_labels_e[u] + 1L, true_labels_e[v] + 1L] <-
      comm_cooc[true_labels_e[u] + 1L, true_labels_e[v] + 1L] + cooc[u, v]
comm_cooc <- comm_cooc / rowSums(comm_cooc)

image(t(comm_cooc[k_emb:1, ]), col = hcl.colors(20, "Blues", rev = TRUE),
      xaxt = "n", yaxt = "n",
      xlab = "Landing community", ylab = "Source community",
      main = "Random Walk Co-occurrence\n(diagonal = within-community)")
axis(1, at = seq(0, 1, length.out = k_emb), labels = paste0("C", 0:(k_emb-1)))
axis(2, at = seq(0, 1, length.out = k_emb), labels = paste0("C", (k_emb-1):0), las = 1)
```

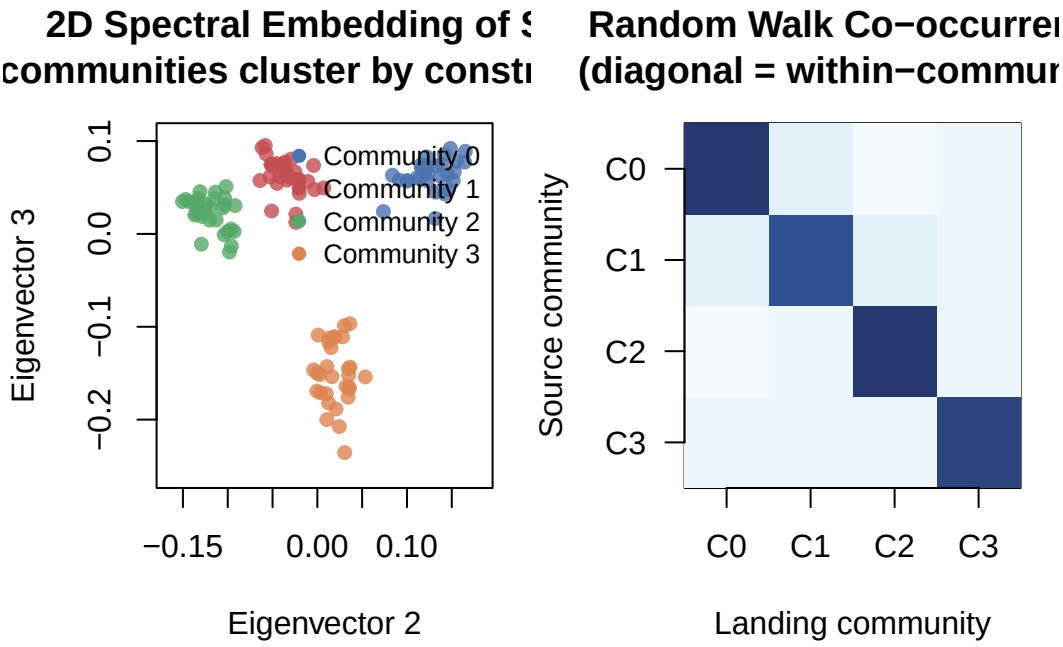

Figure 4: Spectral embedding of an SBM. Communities cluster clearly in 2D.

```
# Spectral embedding similarity: within vs cross community
within_sim <- c(); cross_sim <- c()
for (i in seq_len(n_emb)) {
  for (j in seq_len(n_emb)) {
    if (j <= i) next
    s <- sum(emb[i, ] * emb[j, ])
    if (true_labels_e[i] == true_labels_e[j]) within_sim <- c(within_sim, s)
    else                                      cross_sim  <- c(cross_sim,  s)
```

```
  }
}
cat(sprintf("Spectral embedding similarity:\n"))
```

```
Spectral embedding similarity:
```

```
cat(sprintf("  Within-community: mean = %.4f\n",  mean(within_sim)))
```

```
  Within-community: mean = 0.0158
```

```
cat(sprintf("  Cross-community:  mean = %.4f\n",  mean(cross_sim)))
```

```
  Cross-community:  mean = -0.0053
```

```
cat(sprintf("  Separation ratio: %.2fx\n",
            mean(within_sim) / abs(mean(cross_sim))))
```

```
  Separation ratio: 3.00x
```

5.5 Limitations of Shallow Embeddings

All methods so far produce *transductive* embeddings: one vector per node, learned on a specific graph. They cannot:

- Embed new nodes without retraining
- Incorporate rich node features (text, demographics, sensor readings)
- Adapt to different downstream tasks (same embedding used for both link prediction and community detection)

This motivates **inductive** methods; specifically Graph Neural Networks.

---

6 Graph Neural Networks (GNNs)

6.1 The Core Idea: Learned Neighbourhood Aggregation

A GNN replaces the fixed aggregation of shallow methods with a *learnable* neighbourhood aggregation function:

$$h_v^{(l+1)} = \sigma\Big(W^{(l)} \cdot \mathrm{AGG}\Big(\{h_u^{(l)} : u \in \mathcal{N}(v)\}\Big) + B^{(l)} h_v^{(l)}\Big)$$

where: - $h_v^{(0)} = x_v$ (node features, or a one-hot vector if none exist) - $W^{(l)}, B^{(l)}$ are learnable weight matrices, **shared across all nodes** - AGG is an aggregation function (mean, sum, max, attention-weighted) - σ is a non-linearity (ReLU, etc.)

The sharing of weights across nodes is what makes GNNs **inductive**: the same function is applied to every node's neighbourhood, so it generalises to unseen nodes with the same feature distribution.

6.2 Graph Convolutional Network (GCN, Kipf & Welling 2017)

The most widely used variant simplifies to:

$$H^{(l+1)} = \sigma\Big(\hat{A} H^{(l)} W^{(l)}\Big)$$

where $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ is the symmetrically normalised adjacency with added self-loops ($\tilde{A} = A + I$).

Why self-loops? Without them, $H_v^{(l+1)}$ depends only on neighbours, not on v itself. The self-loop ensures each node *aggregates its own features* together with its neighbours'.

Why symmetric normalisation? It prevents the embeddings of high-degree nodes from having artificially large magnitudes and makes gradient flow more stable. It also corresponds to the Laplacian smoothing filter.

Two-layer GCN for node classification (fully written out):

$$Z = \mathrm{softmax}\Big(\hat{A} \cdot \mathrm{ReLU}\Big(\hat{A} X W^{(0)}\Big) W^{(1)}\Big)$$

Train by minimising cross-entropy on labelled nodes: $\mathcal{L} = -\sum_{i \in V_L} \sum_k y_{ik} \ln Z_{ik}$

6.3 GraphSAGE: Scalable Inductive Learning

GCN requires the full adjacency matrix $\hat{A}$ during training. For a graph with 10^6 nodes, $\hat{A}$ is a $10^6 \times 10^6$ matrix; impossible to store or multiply with.

GraphSAGE (Hamilton, Ying & Leskovec 2017) introduces *neighbourhood sampling*: for each node, sample a fixed-size subset of S neighbours instead of using all of them. This allows mini-batch training:

$$h_v^{(l+1)} = \sigma\Big(W^{(l)} \Big[h_v^{(l)} \,\|\, \mathrm{AGG}\Big(\{h_u^{(l)} : u \in \tilde{\mathcal{N}}_S(v)\}\Big)\Big]\Big)$$

(where $\|$ denotes concatenation and $\tilde{\mathcal{N}}_S(v)$ is a sampled neighbourhood of size S).

Because the weight matrices are *shared*, a trained GraphSAGE model can embed nodes that were not present during training; this is the inductive property.

6.4 Graph Attention Networks (GAT, Veličković et al. 2018)

Instead of uniform (mean) or fixed-normalisation (GCN) aggregation, GAT learns **attention weights** α_{ij} for each edge:

$$\alpha_{ij} = \frac{\exp\big(\mathrm{LeakyReLU}(a^\top [Wh_i \| Wh_j])\big)}{\sum_{k \in \mathcal{N}(i)} \exp(\mathrm{LeakyReLU}(a^\top [Wh_i \| Wh_k]))}$$

$$h_i^{\text{new}} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j\right)$$

Attention weights are interpretable: a large α_{ij} means node i relies heavily on node j for its prediction.

Multi-head attention: run K independent attention heads and concatenate (or average) the results. Reduces variance and captures diverse neighbourhood patterns.

6.5 The Message-Passing Framework

All GNN variants fit a single **message passing neural network (MPNN)** framework (Gilmer et al. 2017):

1. **Message**: $m_{ij}^{(l)} = \phi\big(h_i^{(l)}, h_j^{(l)}, e_{ij}\big)$
2. **Aggregate**: $a_i^{(l)} = \bigoplus_{j \in \mathcal{N}(i)} m_{ij}^{(l)}$
3. **Update**: $h_i^{(l+1)} = \gamma\big(h_i^{(l)}, a_i^{(l)}\big)$

The choice of ϕ, $\bigoplus$, and γ defines the GNN variant:

Model	$\bigoplus$	Notes
GCN	Normalised sum	Fixed weights, simple
GraphSAGE	Mean / max	Sampled, inductive
GAT	Attention-weighted sum	Learnable weights
GIN	Sum	Most expressive (WL-equivalent)

GIN (Xu et al. 2019) proves that sum aggregation is necessary for maximum expressiveness: mean and max aggregations can fail to distinguish different graph structures that sum can tell apart.

```
build_a_hat <- function(A_dense) {
  n       <- nrow(A_dense)
  A_tilde <- A_dense + diag(n)
  d       <- rowSums(A_tilde)
  d_isqrt <- ifelse(d > 0, 1 / sqrt(d), 0)
  D_isqrt <- diag(d_isqrt)
  D_isqrt %*% A_tilde %*% D_isqrt
}

gcn_forward <- function(A_hat, X, W1, W2) {
  # Layer 1: ReLU(A_hat %*% X %*% W1)
  H1 <- A_hat %*% X %*% W1
  H1[H1 < 0] <- 0
  # Layer 2 + softmax
  Z  <- A_hat %*% H1 %*% W2
  Z  <- Z - apply(Z, 1, max)          # numerical stability
  Z  <- exp(Z)
  Z  / rowSums(Z)
}
```

```r
set.seed(42)
k_gcn <- 3L; n_g <- 30L; n_gcn <- k_gcn * n_g
pref_g <- matrix(0.02, k_gcn, k_gcn); diag(pref_g) <- 0.3
g_gcn  <- sample_sbm(n_gcn, pref = pref_g, block.sizes = rep(n_g, k_gcn))
true_labels_g <- rep(seq_len(k_gcn) - 1L, each = n_g)

A_gcn <- as.matrix(as_adjacency_matrix(g_gcn, sparse = FALSE))
A_hat <- build_a_hat(A_gcn)

# Class-informative node features
means_g <- matrix(rnorm(k_gcn * 8) * 2, nrow = k_gcn)
X_gcn   <- means_g[true_labels_g + 1L, ] + matrix(rnorm(n_gcn * 8), n_gcn, 8)

# Random (untrained) weights
W1_gcn <- matrix(rnorm(8 * 16) * 0.1, 8, 16)
W2_gcn <- matrix(rnorm(16 * k_gcn) * 0.1, 16, k_gcn)

Z_gcn <- gcn_forward(A_hat, X_gcn, W1_gcn, W2_gcn)
cat(sprintf("GCN forward pass (untrained weights):\n"))
```

```
GCN forward pass (untrained weights):
```

```r
cat(sprintf("  Input features:    X shape = (%d, %d)\n",  nrow(X_gcn),  ncol(X_gcn)))
```

```
  Input features:    X shape = (90, 8)
```

```r
cat(sprintf("  After layer 1:     H1 shape = (%d, 16)\n", n_gcn))
```

```
  After layer 1:     H1 shape = (90, 16)
```

```r
cat(sprintf("  Output (logits):   Z shape = (%d, %d)\n",  nrow(Z_gcn),  ncol(Z_gcn)))
```

```
  Output (logits):   Z shape = (90, 3)
```

```r
cat("\nPredicted class probabilities for first 5 nodes:\n")
```

```
Predicted class probabilities for first 5 nodes:
```

```
for (i in seq_len(5)) {
  cat(sprintf("  Node %d (true class %d): %s\n",
              i - 1L, true_labels_g[i],
              paste(round(Z_gcn[i, ], 3), collapse = "  ")))
}
```

```
  Node 0 (true class 0): 0.295  0.353  0.352
  Node 1 (true class 0): 0.295  0.351  0.354
  Node 2 (true class 0): 0.307  0.348  0.344
  Node 3 (true class 0): 0.3  0.346  0.354
  Node 4 (true class 0): 0.298  0.351  0.351
```

```
# Effect of aggregation depth
cat("\n--- Effect of aggregation depth ---\n")
```

```
--- Effect of aggregation depth ---
```

```
for (depth in 0:2) {
  X_smooth <- X_gcn
  for (d in seq_len(depth)) X_smooth <- A_hat %*% X_smooth
  within_var <- var(X_smooth[true_labels_g == 0, 1])
  cross_var  <- var(sapply(0:(k_gcn - 1), function(cc)
    mean(X_smooth[true_labels_g == cc, 1])))
  cat(sprintf("  Depth %d: within-class var=%.3f, between-class spread=%.3f, ratio=%.2f\n",
              depth, within_var, cross_var, cross_var / max(within_var, 1e-9)))
}
```

```
  Depth 0: within-class var=1.077, between-class spread=1.146, ratio=1.06
  Depth 1: within-class var=0.158, between-class spread=0.842, ratio=5.35
  Depth 2: within-class var=0.060, between-class spread=0.670, ratio=11.12
```

6.6 Practical Checklist: When to Use What

```
 Question
 Homophilic or heterophilic?
 Transductive or inductive?
 Node features available?
 How many GNN layers?
 Interpretability needed?
```

```
Scale > 1M nodes?
Overlapping communities?
Recommendation
Homophilic → GCN/GAT; Heterophilic → GraphSAGE or H2GCN
Transductive → GCN (full graph); Inductive → GraphSAGE
Yes: use them as x_v; No: use degree or identity matrix
2-3 layers (deeper → over-smoothing, all nodes same vector)
GAT (attention weights); GNN-Explainer for post-hoc
Neighbour sampling (GraphSAGE, ClusterGCN, SIGN)
NMF or BigCLAM; hard-partition methods miss overlaps
```

! Over-smoothing: the depth problem

With L GNN layers, each node aggregates information from its L-hop neighbourhood. For $L \gg 1$, every node's representation converges to the same vector (proportional to the dominant eigenvector of $\hat{A}$). This *over-smoothing* destroys the discriminative signal. In practice: **2 layers suffices for most tasks**. If deeper is needed, use skip connections (jumping knowledge networks) or residual connections.

---

7 Summary: ML on Networks

7.1 The Three-Problem Framework

Task	Simplest method	Best ML method
Community detection	Louvain (modularity)	Leiden / DC-SBM inference
Node classification	Label propagation	GCN / GAT
Link prediction	Common neighbours	Katz + GNN (dot product)
Node embedding	Spectral (Laplacian eigenmaps)	Node2Vec / GraphSAGE

Key assumption	Main failure mode
Modularity > 0	Resolution limit
Homophily	Heterophily
Triangle closure	Degree bias
Proximity ~ similarity	Transductive only

7.2 What Each Method Actually Optimises

Method	Objective
Modularity (Louvain)	$\max_z \sum_c [m_c/m - (\kappa_c/2m)^2]$
SBM MLE	$\max_{z,M} \ln \mathcal{L}(G \mid z, M)$
Label propagation	Fixed point of $F = D^{-1}AF$ (with clamping)
GCN	$\min_W \mathcal{L}_{\text{CE}}(Z[V_L], y[V_L])$
DeepWalk / Node2Vec	$\max_Z \sum_{(u,v)\in\text{walks}} \log \sigma(z_u^\top z_v)$
Katz link prediction	$s(i,j) = [(I - \beta A)^{-1} - I]_{ij}$

7.3 References

- Blondel, V. D. et al. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, P10008.
- Traag, V. A., Waltman, L. & van Eck, N. J. (2019). From Louvain to Leiden. *Scientific Reports*, 9, 5233.
- Zhu, X. & Ghahramani, Z. (2002). Learning from labeled and unlabeled data with label propagation. *CMU Technical Report* CMU-CALD-02-107.
- Adamic, L. A. & Adar, E. (2003). Friends and neighbors on the web. *Social Networks*, 25(3), 211–230.
- Perozzi, B., Al-Rfou, R. & Skiena, S. (2014). DeepWalk. *KDD 2014*.
- Grover, A. & Leskovec, J. (2016). Node2Vec. *KDD 2016*.
- Kipf, T. N. & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *ICLR 2017*.
- Hamilton, W. L., Ying, R. & Leskovec, J. (2017). Inductive representation learning on large graphs. *NeurIPS 2017*.
- Veličković, P. et al. (2018). Graph attention networks. *ICLR 2018*.
- Xu, K. et al. (2019). How powerful are graph neural networks? *ICLR 2019*.
- Gilmer, J. et al. (2017). Neural message passing for quantum chemistry. *ICML 2017*.
- Newman, M. E. J. (2013). *Networks: An Introduction*. Ch. 11–14.